

Proyecto Final - Sistema RAG

Introducción a la Inteligencia Artificial

Zeus Sarmiento

02/10/2026

Proyecto Final - RAG

Este proyecto es un prototipo de contratos con RAG, montado en AWS, usando S3 como origen de los documentos y como base vectorial (*Amazon S3 Vectors* + *Amazon Bedrock*). El enunciado pide el mismo ciclo en local con Streamlit, FastAPI, Chroma y Google AI. El enfoque no cambió: incrustar, indexar, recuperar los k más cercanos y recién entonces generar. Cambiaron las herramientas, para probar ese ciclo en la nube en el ecosistema de AWS.

MiCasa arma borradores de contratos de vivienda en Yucatán: renta o compraventa (la idea sería contratos en general; este recorte es para acotar el alcance). Hablas con un agente, el sistema escribe o actualiza el documento, y también se puede editar a mano, como en Word. Se baja en Word o PDF.

Un modelo “sabe” español, pero no tiene el Código Civil de Yucatán como fuente de verdad (así como otros documentos legales relevantes) y mezcla artículos. Por eso el RAG: primero busca en textos que nosotros subimos, y después responde o redacta con eso, no de memoria.

Hay dos puertas al RAG. El **asistente legal** (el círculo) es el de las evidencias: preguntas libres, Retrieve top-5, respuesta con fuentes y score; no toca el contrato. Y el editor de **documentos**, que también consulta el índice cuando pides una cláusula que no está en plantilla. El cuerpo del contrato (comparecencia, renta, firmas) sale de JSON fijos; el RAG únicamente consulta los documentos legales para justificaciones y referencias.

- **Login:** `admin / Admin2026!`
- **App:** <https://main.d1hsj6zus1f46.amplifyapp.com>.
- **API:** <https://j9zrekl9g4.execute-api.us-east-1.amazonaws.com>.
- Código en `proyecto_final/` (README, `requirements.txt`, `.env.example`).
- Corpus: `data/README.md`.

Cómo se cubre lo que pide el curso

- **Interfaz (Streamlit).** Aquí es React (Vite en local, Amplify en la nube). El usuario pregunta y ve la respuesta con citas. La UI no llama al índice ni al modelo: solo hace HTTP.
- **API (FastAPI: `/health`, `/ingest`, `/query`).** Aquí es API Gateway + Lambda. Mismas tres rutas: `GET /health`, `POST /query` (busca y genera) y `POST /ingest` (no reindexa en el POST: se sube el PDF a S3 y se lanza Sync, que es el ingest real).
- **Índice persistente (Chroma en disco).** Aquí es la Knowledge Base `Q1UNTTTE8X` sobre `s3://micasa-kb-source/`. Reiniciar Lambda no borra el índice. Cada fragmento trae origen y score.
- **Embeddings (Google AI).** Los calcula Bedrock al sincronizar la KB. Pregunta y documentos quedan en el mismo espacio vectorial.
- **Generación (Gemini).** Bedrock Converse, **después** de Retrieve. Modelo: Kimi K3; respaldo Claude 3.5 Haiku. Español, citas, y si no hay evidencia se abstiene.
- **Corpus ≥ 5 docs de un dominio.** Cinco PDFs de vivienda / derecho civil de Yucatán (abajo).
- **No secretos en git.** `.env.example` solo trae `VITE_API_URL`.

- Curso (local) Este prototipo (AWS)
- Streamlit → React
- FastAPI → API Gateway + Lambda
- Chroma → Knowledge Base / S3 Vectors
- Google embeddings → embeddings al Sync de la KB
- Gemini → Kimi K3 / Haiku
- Región `us-east-1`. $k = 5$.

Reporte

El dominio es derecho civil inmobiliario de Yucatán, sobre todo arrendamiento y compraventa de vivienda. Un solo tema, no un cajón de textos sueltos. Alimenté el índice con cinco documentos distintos: el Código Civil del Estado (~3.4 MB, la fuente principal de ley; arrendamiento alrededor de los arts. 1564–1650, compraventa 1397–1473), un formato de arrendamiento, un contrato de estacionamiento, un modelo CANADEVI de preventiva y un contrato de adhesión de PROFECO. Viven en el bucket de origen de la Knowledge Base. El Código Civil se puede bajar de https://www.yucatan.gob.mx/docs/pot/secogey/12_DAJYSP/2022/Fraccion_I/Codigo_Civil_del_Estado_de_Yucatan.pdf (también está en el sitio del Congreso). Las plantillas JSON del contrato no cuentan como corpus: son la forma del documento, no la ley. Al preguntar, pido $k = 5$. Los embeddings los calcula Bedrock cuando sincroniza la Knowledge Base, para que pregunta y documentos queden en el mismo espacio. Para generar uso Kimi K3 (`us.moonshotai.kimi-k3`), con Claude 3.5 Haiku de respaldo. Elegí estos modelos porque son buen balance entre resultados y precio, pero las respuestas podrían mejorar sustancialmente con el uso de un modelo más potente.

No partí los PDFs a mano ni corrí un script de chunks en la laptop. Cuando la Knowledge Base sincroniza, Bedrock los fragmenta (tamaño fijo, con overlap) y les deja metadatos de origen —de qué archivo salió cada trozo—. Lo dejé administrado a propósito: si uno corta a ciegas por número de caracteres, un artículo del Código queda a la mitad, *Retrieve* trae fragmentos sueltos y el modelo empieza a citar de oído. Pedir cinco resultados alcanza para traer el artículo y un poco de lo que tiene al lado, sin llenar el contexto de ruido.

Para no inventar ley, el asistente tiene orden de usar solo lo que se recuperó (abstención). Si ese contexto no alcanza, lo dice y no fabrica artículos ni cifras. Eso importa porque una misma palabra puede vivir en dos títulos distintos. “Depósito”, en el Código de Yucatán, es la fianza de una renta (arts. 1619-1620) y también el contrato de guarda de bienes; si la pregunta es de vivienda, se tiran los chunks del otro tema. Cuando *Retrieve* no trae nada útil, la respuesta cae a que no se pudo con las fuentes. No se rellena con lo que el modelo “recuerda”. Eso es lo que muestra la pregunta fuera de dominio (**Bitcoin, día 32**).

En el diseño del curso, Google AI hace dos papeles distintos —embeber los documentos y generar la respuesta con Gemini— y Chroma es el cajón donde viven los vectores y se hace el k -NN. **Esos papeles están; cambiaron las herramientas.** Embeber lo hace el Sync de la Knowledge Base sobre los PDFs de S3. El cajón y el vecino más cercano los hace *Retrieve*: el índice no se borra si se reinicia la Lambda, pide los cinco más cercanos y cada uno trae score y el nombre del archivo. Generar lo hace *Converse* después de buscar, en español, anclado a esos trozos. No se concatenan los chunks

como si fueran la respuesta, ni se deja que Bedrock escriba el contrato entero. Buscar primero, generar después: igual que en el enunciado.

Estructura del proyecto y equivalencias entre la arquitectura solicitada y las herramientas usadas: https://github.com/Zeusjonass/maestria-ia/blob/main/proyecto_final/docs/estructura-del-proyecto.md

Figuras

Figura 1 — Interfaz: respuesta con citas y scores (lo que el curso pide ver en Streamlit)
En un contrato de renta de casa habitación en Yucatán, ¿cuánto depósito o fianza se puede pedir?
(Debería mencionar el **depósito en garantía / art. 1619**).



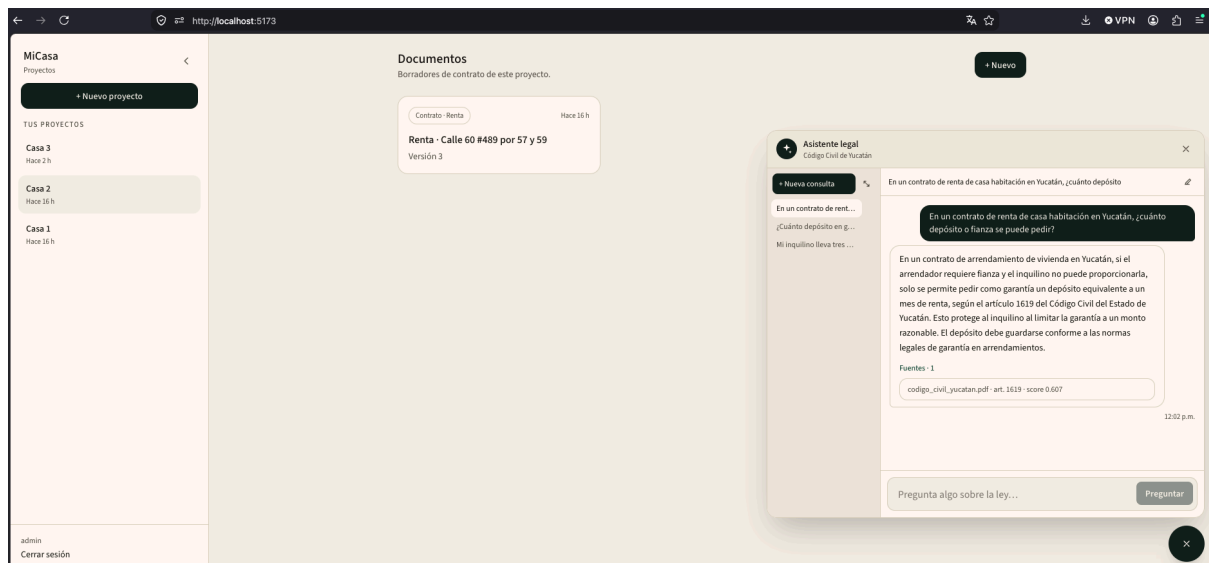


Figura 2 — La misma pregunta por la API (lo que el curso pide contra FastAPI / curl)

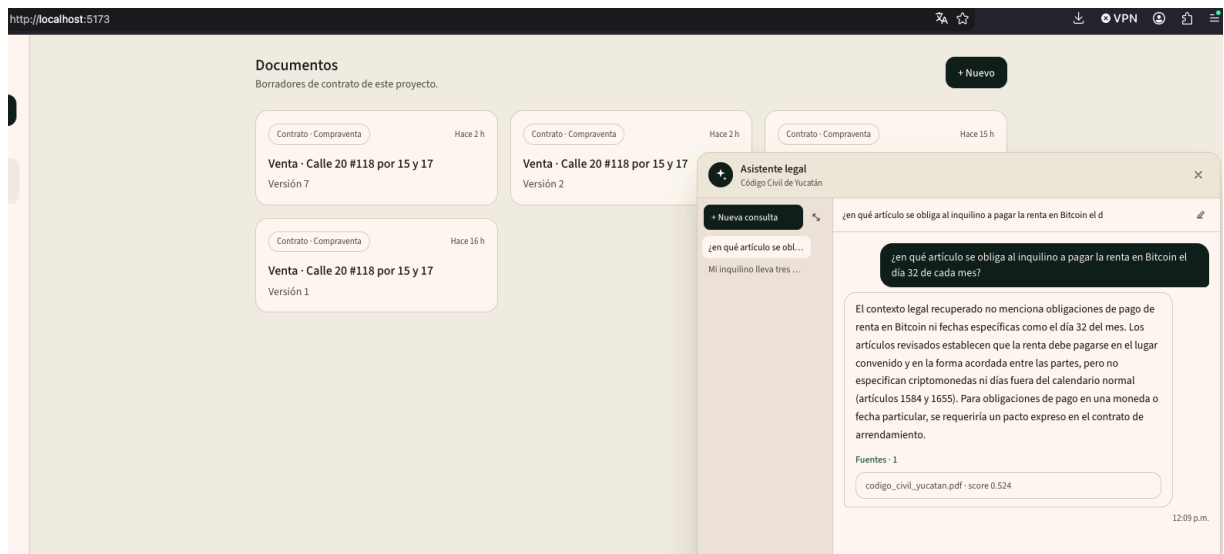
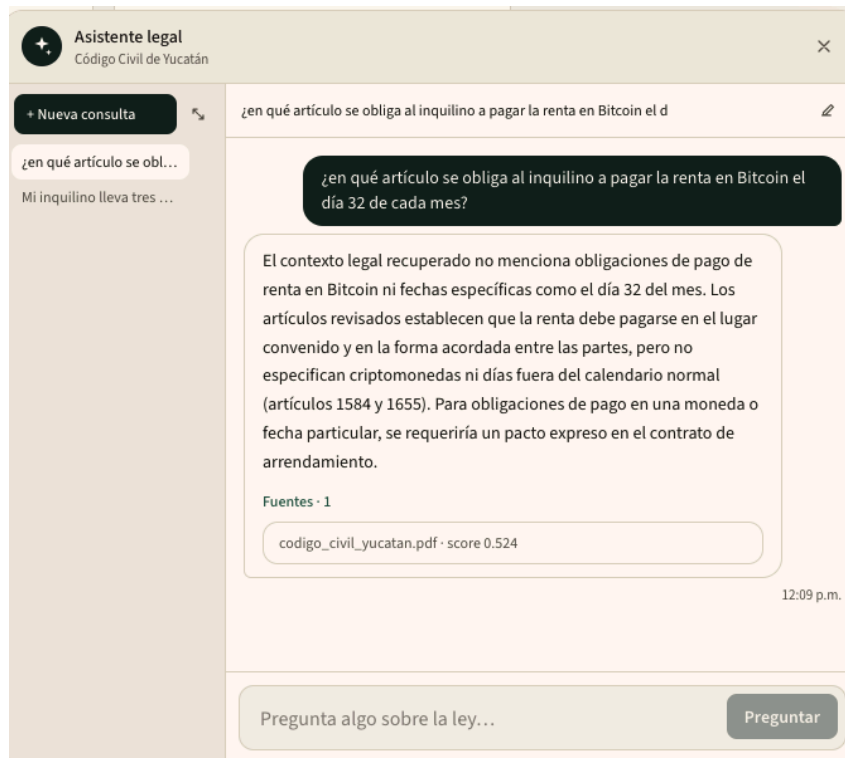
API=https://j9zrekl9g4.execute-api.us-east-1.amazonaws.com

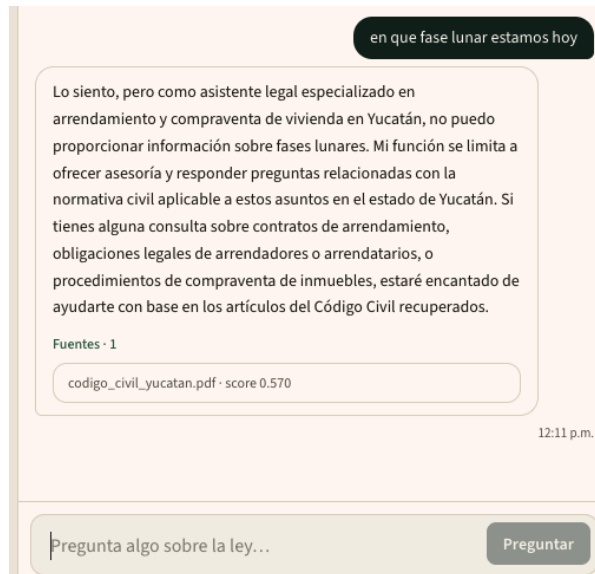
```
curl -s "API/health"curl -s -XPOST"API/query" -H "Content-Type: application/json" \
-d '{"question":"En un contrato de renta de casa habitación en Yucatán, ¿cuánto depósito o
fianza se puede pedir?"}'
```

```
zeussarmiento@Mac-mini-de-Zeus micasa % API=https://j9zrekl9g4.execute-api.us-east-1.amazonaws.com
curl -s "$API/health"
curl -s -X POST "$API/query" -H "Content-Type: application/json" \
-d '{"question":"En un contrato de renta de casa habitación en Yucatán, ¿cuánto depósito o fianza se puede pedir?"}'
{"ok": true, "service": "micasa-chat", "index": "bedroc", "kbId": "Q1UNTTTE8X", "model": "us.m", "el": "us.anthropic.clau", "ast-1"}{"question": "En un contrato de renta de casa habitación en Yucatán, ¿cuánto depósito o fianza se puede pedir?", "answer": "En un contrato de arrendamiento de vivienda en Yucatán, si el arrendador requiere fianza y el inquilino no puede proporcionarla, solo se permite pedir como garantía un depósito equivalente a un mes de renta, según el artículo 1619 del Código Civil del Estado de Yucatán. Esto protege al inquilino al limitar la garantía a un monto razonable. El depósito debe guardarse conforme a las normas legales de fianza en arrendamiento.", "citations": [{"source": "codigo_civil_yucatan.pdf", "article": "1619", "score": 0.6072}], "retrieved": [{"source": "codigo_civil_yucatan.pdf", "score": 0.6072}, {"source": "codigo_civil_yucatan.pdf", "score": 0.4155}, {"source": "codigo_civil_yucatan.pdf", "score": 0.391}, {"source": "codigo_civil_yucatan.pdf", "score": 0.389}], "k": 5, "retrievedCount": 4, "abstained": false, "metrics": {"latencyMs": 2067, "retrievalMs": 620}}
zeussarmiento@Mac-mini-de-Zeus micasa %
```

Figura 3 — Preguntas fuera de dominio

- ¿En qué artículo se obliga al inquilino a pagar la renta en Bitcoin el día 32 de cada mes?
- ¿Cuál es la fase lunar de hoy?





Video Demo:

<https://drive.google.com/file/d/10zE1V1euM1KhaGOmKMVgDYIZYtnC--1J/view?usp=sharing>

Repositorio: https://github.com/Zeusjonass/maestria-ia/tree/main/proyecto_final

Las rutas del API (equivalente a **/docs** de FastAPI):

<https://j9zrek19g4.execute-api.us-east-1.amazonaws.com/docs>

Spec: <https://j9zrek19g4.execute-api.us-east-1.amazonaws.com/openapi.json>